

Latency Autopsy

VisionPsy-Nano-460M takes 99–111 seconds to read one document. Gemma 4, a model 6–8× larger, answers in well under a minute on the same phone. This traces exactly where every second goes, on both sides, and why the smaller model isn't the one that's slow.

Device	Chipset
Samsung Galaxy M35 5G · SM-M356B	Exynos 1380 (s5e8835) · Mali-G68 · 8 cores
Memory	Note
5.6 GB total · Android 16	Same physical unit for both apps – confirmed via adb

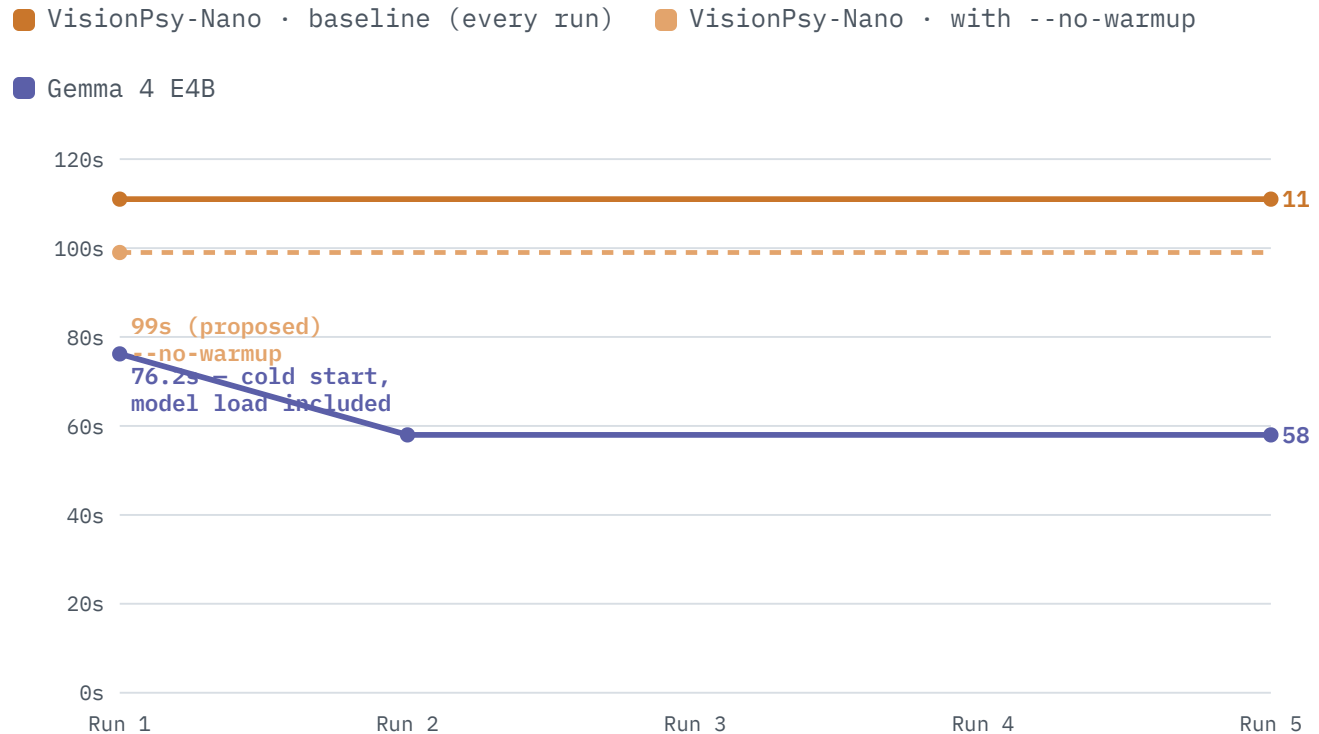
THE HEADLINE NUMBERS

111s Our current baseline, every single extraction — PAN card, CPU, 4 threads	92% Of that 111s spent before the model outputs one real token of the answer	99s Measured after removing one redundant step (- -no-warmup) — not yet shipped	58s Gemma 4 E4B, 6× our combined model size, per image once it's loaded
---	--	---	---

It isn't model size

The instinct is that a 460M-parameter model should be faster than a model with an effective 2–4B, real 8B parameter count. On this phone, measured on the same silicon, it's the opposite: Gemma's *worst case* — a cold app launch plus its first image — still lands at roughly 76 seconds, already faster than our *best* measured case. Every image after that first one drops to 58 seconds. Ours never drops. It's 99–111 seconds on the first run, the fifth run, the fiftieth.

That flat line is the actual finding. Model size explains almost none of the gap — how each app manages the model's lifecycle explains nearly all of it.



Per-extraction wall clock across five consecutive uses in one session, same phone, CPU only. VisionPsy-Nano reloads the full model on every run, so the line never moves. Gemma pays its model load once (run 1), then every later image is vision-encode + generation only.

Ranked causes

Ordered by how much of the 111 seconds each one actually accounts for, not by how interesting it sounds.

1

Full process + model reload on every single run MEASURED

`runOnePass()` launches `libllama-mtmd-cli`.so as a brand-new OS process for every tap of Run, passing `-m/-mmproj` fresh each time. That means reading ~310MB (language model) + ~104MB (vision projector) off flash, parsing the GGUF, and rebuilding the whole compute graph and KV cache from nothing — every time. Gemma OCR's `OcrViewModel.preload()` loads its Engine once, in the background, before the user even picks an image, and keeps it resident for the rest of the session.

Accounts for ~102 of 111 seconds (measured via a `-n 1` isolation run – see the breakdown below).

2 No compiled-weight or kernel cache carried between runs DOCUMENTED MEASURED

Google's LiteRT-LM writes XNNPACK weight-prep caches and compiled GPU programs to `filesDir/litert-cache`, which is why Gemma's own load time drops from 18.2s (first load) to 2.1s (cached). Our invocation has no equivalent: because each run is an independent process that starts and exits, there's no long-lived cache to benefit from even if `llama-mtmd-cli` supported one internally — there's no second warm request in the same process to hand it to.

This is the direct answer to "is it caching?" – yes, partly: Gemma has a real one, and we structurally can't use one under the current per-run-process design regardless of what the binary supports.

3 Automatic model warm-up pass, left enabled MEASURED

`llama.cpp`'s `common_init_from_params` runs one full dummy forward pass through the model on every load — visible in the raw log as `warming up the model with an empty run...` (`--no-warmup` to disable) — and we've never passed that flag. Confirmed with a direct A/B test on this device, same image, same prompt, deterministic (`--temp 0`) output.

111s → 99s with `--no-warmup` added. 12 seconds, byte-identical result. Not yet shipped in the app.

4 Runtime engineering gap DOCUMENTED

Gemma OCR runs on Google's own LiteRT-LM 0.17.1 + XNNPACK, a runtime built and hardware-tuned specifically for mobile ARM inference. We run a patched general-purpose `llama.cpp` CLI — solid, and the only way to serve the custom SigLIP2 projector today, but `llama-mtmd-cli` is a one-shot command-line tool by design, never built with a warm, repeated-invocation use case in mind. That design mismatch is baked into every one of the other causes on this list.

Not separately measurable in seconds – it's the reason causes #1-#3 exist at all.

5 Vision-encoder token budget UNCONFIRMED

Gemma's mobile export deliberately caps every image at 280 tokens (visible as `vision_70/140/280` subgraphs in logcat) — a documented speed/accuracy trade that blurs dense text and forces their strip-splitting workaround. We do not have a confirmed token count for our SigLIP2 pipeline to compare against. Flagging this as open rather than claiming it's a cause.

No number to report yet — would need direct instrumentation of the mmproj pass to confirm either way.

TESTED AND RULED OUT

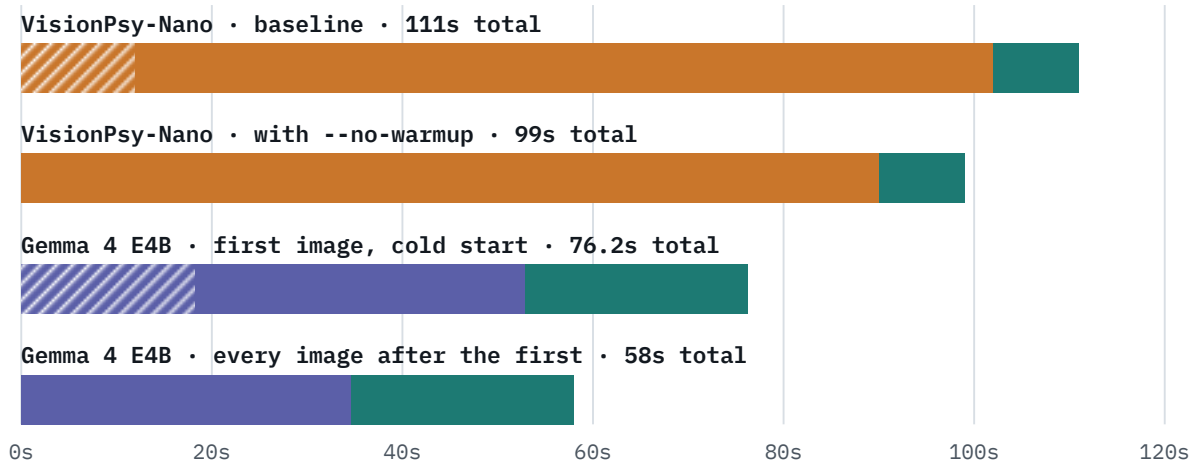
- **Grammar-constrained decoding (-j f).** The stage1 fine-tune runs with grammar entirely disabled and showed no latency change — if anything this also tells us the other four causes are process/model-load bound, not decoding-strategy bound.
- **GPU offload.** Already tested on this exact Mali-G68 and found slower than CPU (no cooperative-matrix support) — see the GPU section below for why Gemma independently reached the same "stay on CPU" conclusion, for a related but distinct reason.
- **Raw parameter count.** Directly contradicted by the evidence: the 6–8× larger model is the faster one here.

Where the 111 seconds actually goes

Derived from three back-to-back runs on this device, same image, same prompt: a normal run (-n 512), a load-only isolation run (-n 1, so generation is negligible), and a normal run with --no-warmup added. The arithmetic: $111 - 102 = 9$ s of real generation; $111 - 99 = 12$ s of warmup; $102 - 12 = 90$ s of load + vision-encode + prefill. Gemma's two rows come straight from its own handoff doc's measured figures.

■ Load + vision-encode + prefill 🗑 Warm-up pass (avoidable)

📄 Model load (one-time only) ■ Generation



All four bars share the same 0–120s scale. Note that our load-and-encode segment alone (90s) already exceeds Gemma's *entire* per-image cost (58s) — the gap isn't in who reads the image faster, it's in how much gets redone before reading starts.

Side by side

DIMENSION	VISIONPSY-NANO-460M (OURS)	GEMMA 4 (E2B/E4B)	EVIDENCE
Runtime	Patched llama.cpp, llama-mtmd-cli (custom SigLIP2 mmproj support)	Google LiteRT-LM 0.17.1, XNNPACK backend	DOCUMENTED
Model size on disk	~310MB LM + ~104MB projector (~415MB combined)	2.59GB (E2B) / 3.66GB (E4B) — single file	MEASURED
Process model	New OS process spawned per extraction, exits after	One persistent Engine, loaded once, reused for the session	MEASURED / DOCUMENTED
First load cost	Paid on <i>every</i> run: ~90s (excl. warmup)	18.2s once, then 2.1s on later cold starts (compiled-cache hit)	MEASURED / DOCUMENTED
Weight-prep / kernel cache	None exposed in our invocation pattern	filesDir/litert-cache, ~2.4GB for E4B CPU	DOCUMENTED

DIMENSION	VISIONPSY-NANO-460M (OURS)	GEMMA 4 (E2B/E4B)	EVIDENCE
Memory mapping	llama.cpp default is mmap-on (we never pass --no-mmap) — inferred, not directly observed in logs	Explicitly mmmaps the .litertlm file; a named design pillar in their own doc	UNCONFIRMED (ours) / DOCUMENTED (theirs)
CPU threads used	4 (half of 8 cores, no big/little awareness)	4 (explicitly reasoned as "the 4 performance cores on Exynos 1380")	DOCUMENTED
Vision token budget	Not measured	Capped at 280 tokens — a deliberate speed/accuracy trade	UNCONFIRMED (ours) / DOCUMENTED (theirs)
Decoding	Greedy (--temp 0), grammar-constrained JSON for 5 of 7 doc types	Greedy (topK=1), free-text, no grammar constraint	DOCUMENTED
GPU on this Mali-G68	Vulkan tested, slower than CPU (no cooperative-matrix)	OOM-killed — non-pageable CPU-side weight copies exhaust 6GB RAM	DOCUMENTED both
Per-image cost once warm	99–111s, unchanged run after run	58s, flat from the second image onward	MEASURED / DOCUMENTED

The GPU dead end, on both stacks

Worth calling out because both teams independently arrived at "stay on CPU" for the same GPU family, by two different roads.

Ours: Vulkan compute offload was built and tested against this Mali-G68 and came back slower than CPU — Mali doesn't support cooperative-matrix instructions, which is what makes Vulkan matrix multiply competitive on GPUs that do. A pure throughput loss, not a crash.

Gemma's: a memory-safety problem, not a throughput one. LiteRT logs `Weights preparation on Gpu is disabled for PowerVR, Broadcom, Mali GPUs`, then builds CPU-side copies of the weights for every compiled graph (decode,

prefill_1024, prefill_128, verify). Those copies are anonymous memory — unlike the mmap'd original, the OS can't page them back out under pressure. Loading E4B on GPU delegated 2,712 decode ops to OpenCL, ran for about two minutes, dropped available memory to ~1GB with ~900MB in swap, and Android's low-memory killer took the app down as the *foreground* process. It left a 2.2GB cache file behind.

Same underlying constraint (this Mali GPU's limitations), two different failure signatures: ours degrades gracefully to "slower," theirs fails catastrophically to "killed." Neither app should turn GPU on by default on this class of hardware.

Status: fixed, proposed, unexamined

MEASURED, NOT YET SHIPPED

Add --no-warmup

One-line addition to `runOnePass()`'s arg list.
Byte-identical output confirmed via `--temp 0` A/B test.

111s → 99s (-12s, -11%)

NEEDS CONFIRMING, NOT CHANGING

Memory mapping

Almost certainly already on (llama.cpp's default; we never pass the disabling flag). Not directly seen in the captured log tail — worth a full-log check, not a code change.

Expected: no action needed

PROPOSED, UNTESTED

Core-aware thread pinning

We use "half of nproc" (4) with no knowledge of which 4 cores those are. Gemma pins to the 4 known performance cores on this exact chip. Same count, different guarantee.

Unknown — needs a live test

LARGE CHANGE · NEEDS A FEASIBILITY CHECK FIRST

Keep the model resident (server mode)

The actual fix for cause #1. Requires confirming the patched build's server binary carries the same custom SigLIP2 mmproj support as `llama-mtmd-cli` before any of this is built — a rewrite of the inference layer, process lifecycle, and Stop/cancel handling, not a flag.

Potential: 111s → ~9-20s per repeat run

Evidence log

Exact commands run on-device for this report, same phone (RZCXA1SXH5V), same image, pan.json schema, 4 threads, CPU only, --temp 0.

Load-only isolation · establishes the load+vision+prefill floor

```
llama-mtmd-cli -m visionpsy-nano-460m-q5_k_m.gguf --mmproj mmproj-visionpsy-nano-460m-q8.gguf \  
-jf pan.json --image vlm_input.jpg -sys "$SYS" -p "$PROMPT" \  
-n 1 --temp 0 -t 4 -tb 4 -ngl 0 --no-mmproj-offload --no-op-offload  
→ 102s
```

Baseline full run · -n 512, everything else identical

```
... -n 512 --temp 0 -t 4 -tb 4 -ngl 0 --no-mmproj-offload --no-op-offload  
→ 111s  
{  
  "document_type": "pan", "pan_number": "SICBX8310Y",  
  "name": "Mohamed Modhar stride", "parent_name": "Ganjikunta Mahash",  
  "parent_relation": "father", "dob": "02/06/1997"  
}
```

Same full run, one flag added

```
... -n 512 --temp 0 -t 4 -tb 4 -ngl 0 --no-mmproj-offload --no-op-offload --no-warmup  
→ 99s (identical JSON output to the run above)
```

The log line that started this thread · printed by common_init_from_params on every load we don't pass --no-warmup to

```
common_init_from_params: warming up the model with an empty run - please wait ...  
(--no-warmup to disable)
```

Gemma-OCR-Developer-Handoff.docx · §2.2, the Mali GPU warning

Weights preparation on Gpu is disabled for PowerVR, Broadcom, Mali GPUs

Gemma-OCR-Developer-Handoff.docx · §6.2, the only clean-run timing figure it reports

E4B on CPU loaded in 18.2s (2.1s once cached) with ~2.5GB RAM still free.
Timing: vision encode plus prefill 34.7s, generation 3.5 tokens/s, total 58s.

SOURCES

- docs/MODEL_HANDOVER_stage1.md — stage1 fine-tune scope, generation settings, and format-vs-accuracy findings.
- docs/Gemma-OCR-Developer-Handoff.docx — §1–2 (architecture, key concepts), §6 (development lessons, GPU OOM), §9 (test plan). Gemma's own 58s/18.2s/34.7s/280-token figures are quoted directly from here, measured on a clean synthetic invoice, not a real phone photo.
- Live diagnostics, this session — three back-to-back on-device runs (-n 1, baseline, --no-warmup) on device RZCXA1SXH5V, plus `getprop//proc/meminfo` confirming it's the same physical unit as the Gemma doc's test phone.
- MainActivity.kt, runOnePass() / runExtraction() — current per-run process-spawn behavior.